

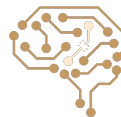
Recap of Session 1 & 2

Arash Marioriyad

16 Farvardin 1405

CE 417
Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188



Last Two Sessions

- Session 1:
 - Introduction to AI
- Session 2:
 - Search Problems
 - Uninformed Search Algorithms



Today: Informed Search

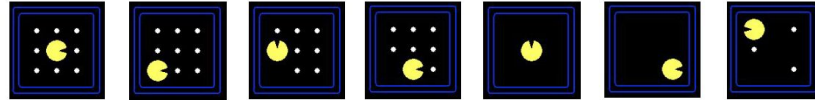
- Heuristic
- Greedy Search
- A^* Search
- Admissible Heuristic
- Graph Search
- Consistent Heuristic



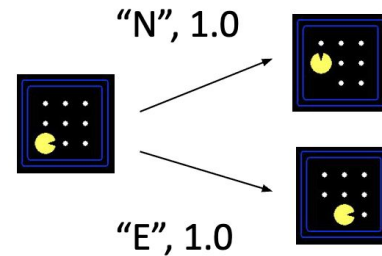
Search Problems

- A **search problem** consists of:

- A state space

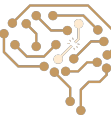


- A successor function
(with actions, costs)

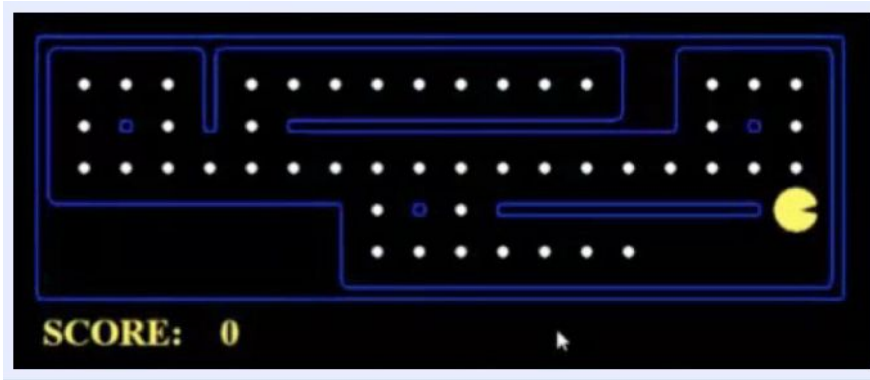


- A start state and a goal test

- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state



Example: Pacman

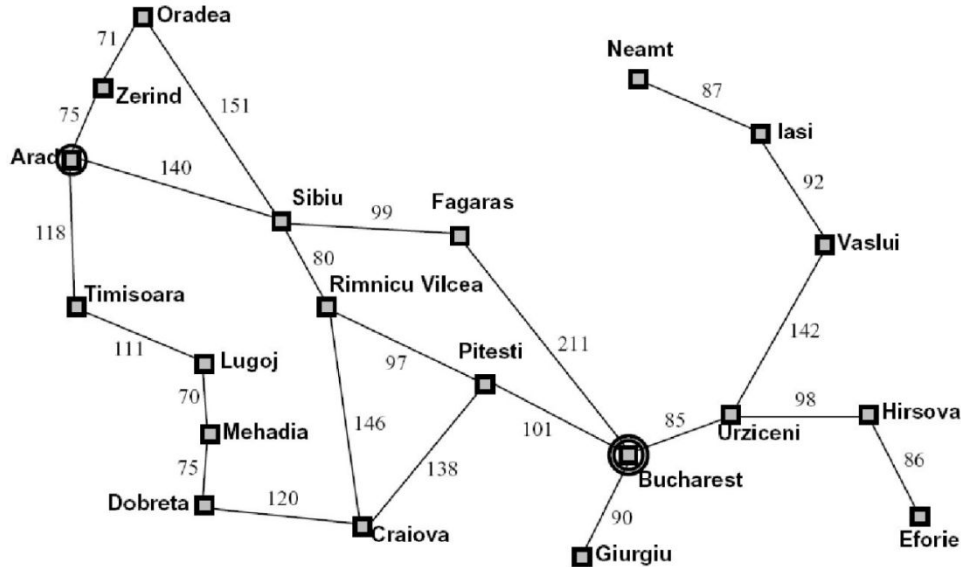


■ Problem: Eat-All-Dots

- States: $\{(x,y), \text{dot booleans}\}$
- Actions: NSEW
- Successor: update location and possibly a dot boolean
- Goal test: dots all false



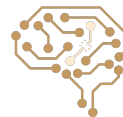
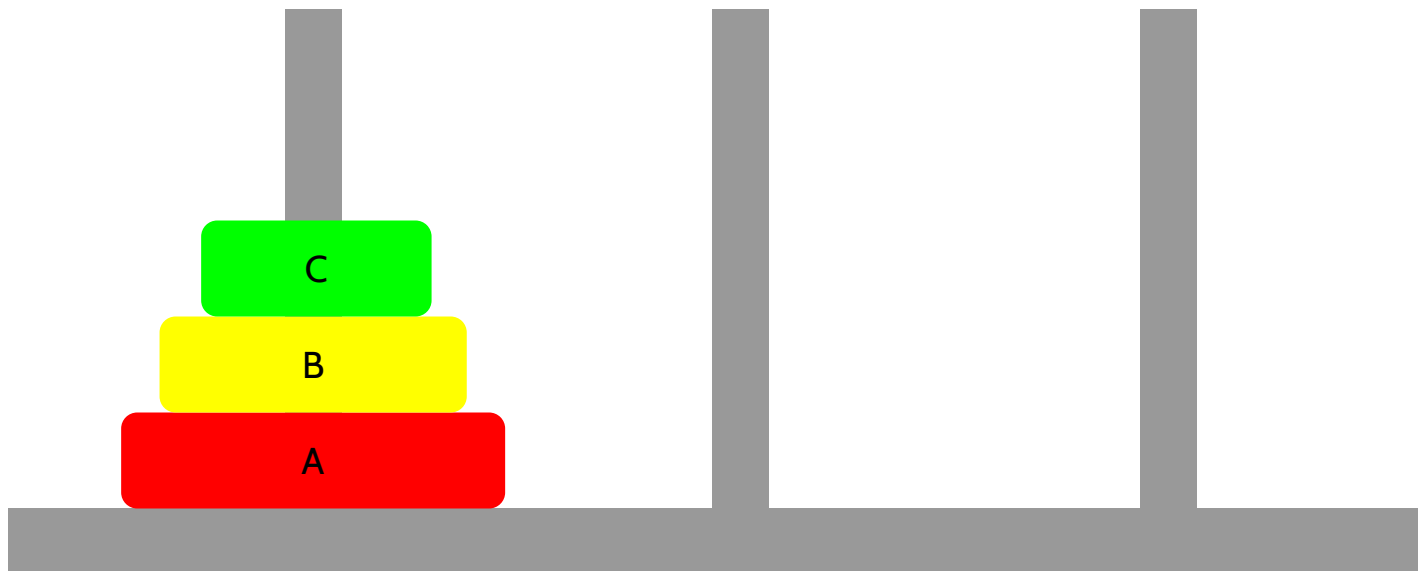
Example: Traveling in Romania



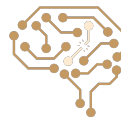
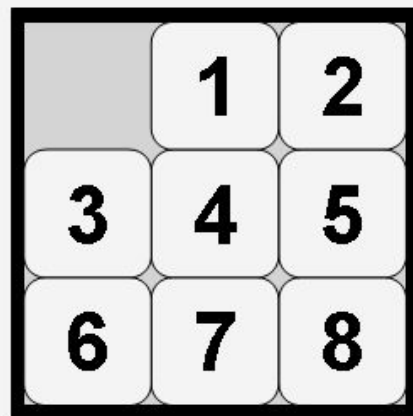
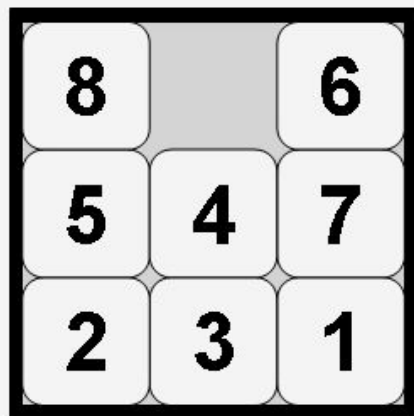
- State space:
 - Cities
- Successor function:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?
- Solution?



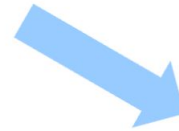
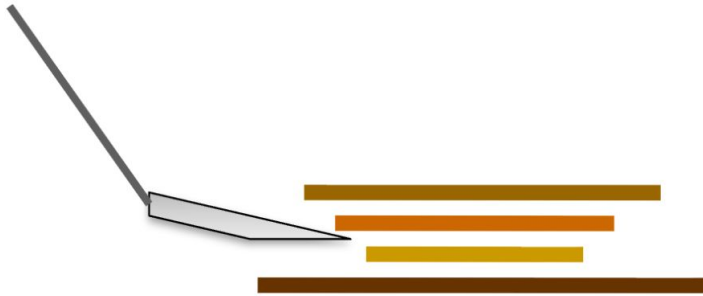
Towers of Hanoi



Eight Puzzle



Pancake Problem



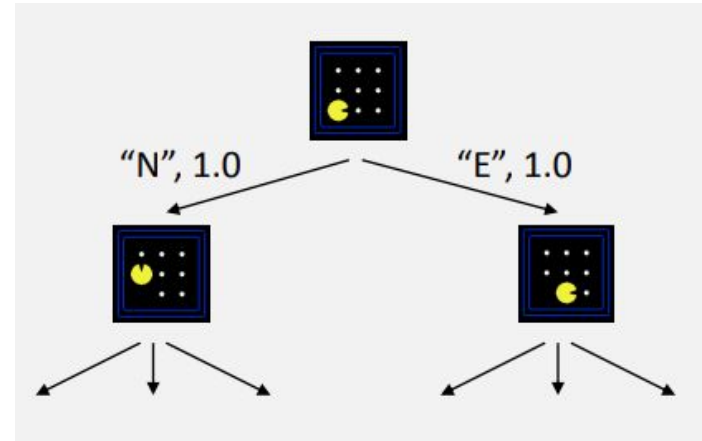
Two Lenses Over Search Problems

- Problem Designer Lens:
 - How should we model the world?
 - What should be the states and their relations?
- Search Algorithm Designer Lens:
 - Which search algorithm to choose?
 - Time and Space Complexity
 - Completeness and Optimality

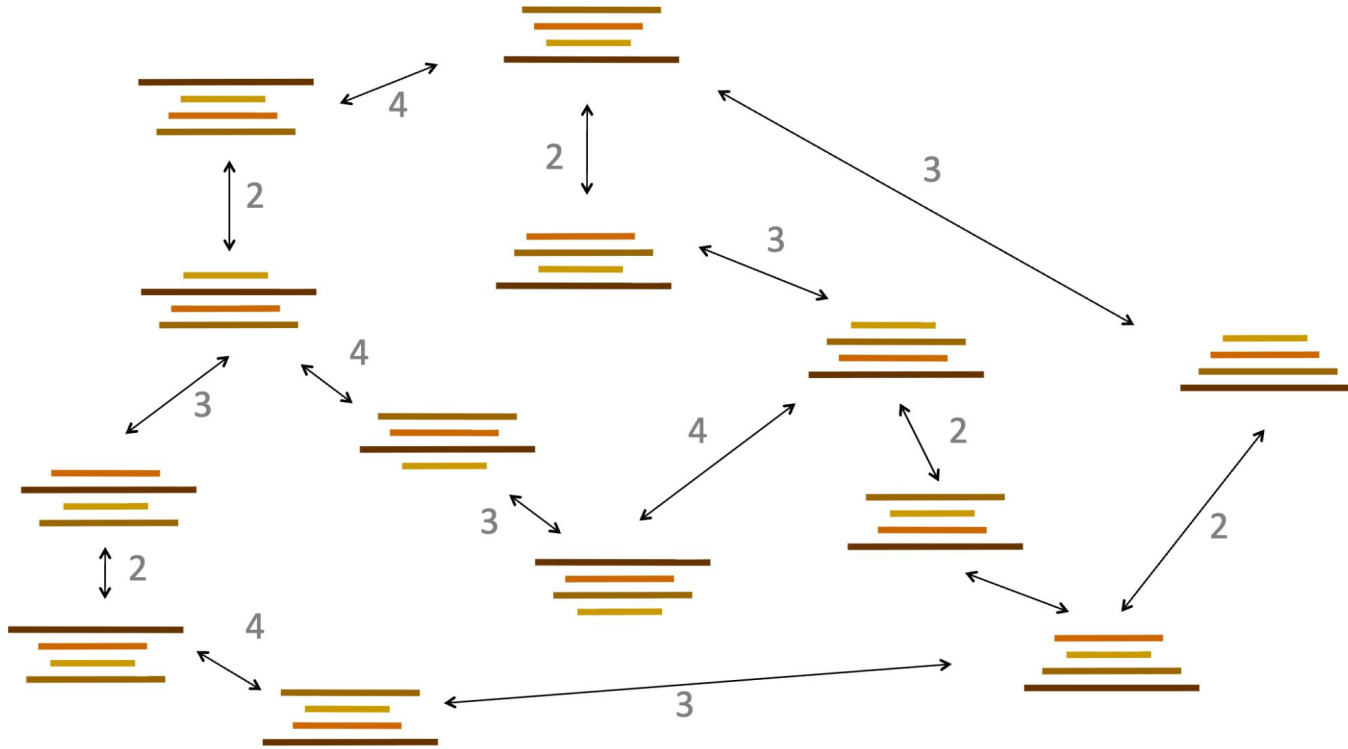


Tree Search

- A general framework for search algorithms
- Nodes are plans (sequence of states obtained by applying actions)
- Plans have costs
- We may see a state multiple times



Pancake Problem - State Space Graph



Search Algorithms

- What do search algorithms tell us?
 - How to build the search tree?
 - How to select the next element (unexplored partial plans) from fringe?
- What don't search algorithms tell us?
 - In what order add the partial plans to fringe?
 - When to apply goal test? expansion time or at **dequeuing time**?



Tree Search

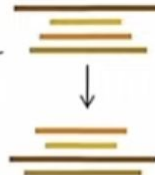
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



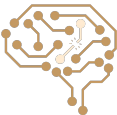
Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Action: flip top two
Cost: 2

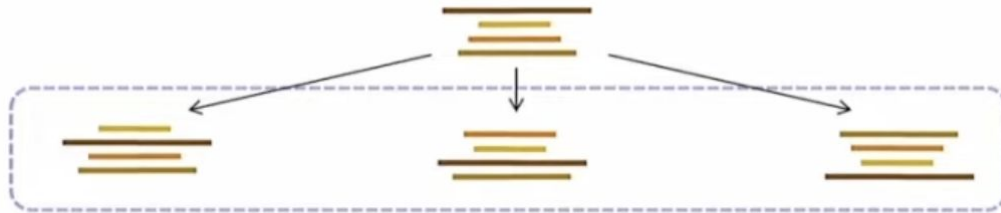


Action: flip all four
Cost: 4



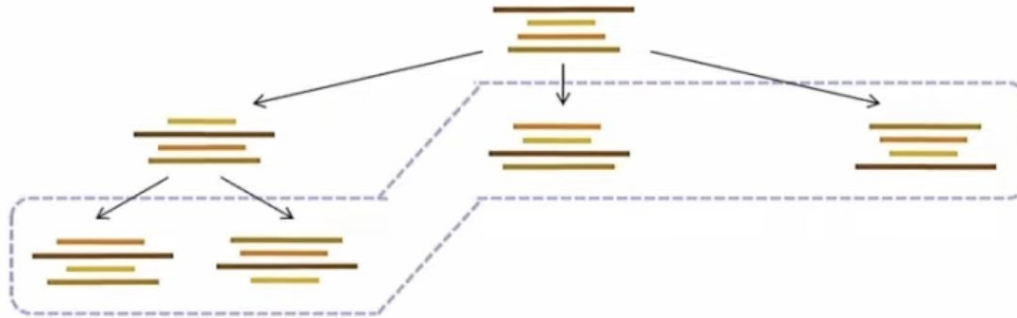
Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



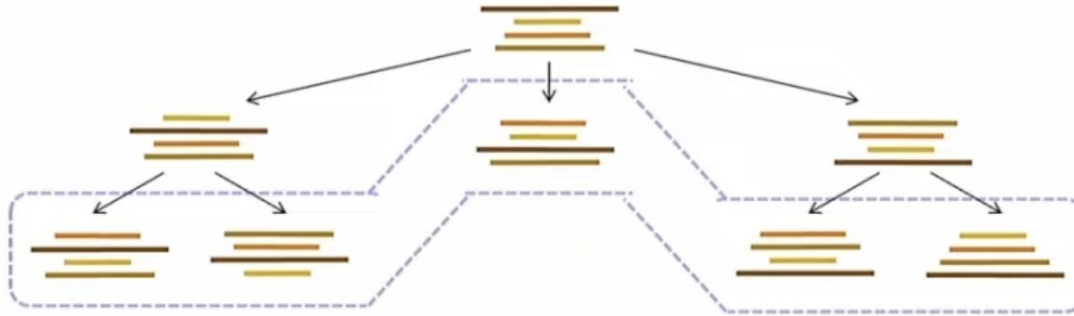
Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



Search Algorithms (cont.)

- **Complete:** If it is guaranteed to find a solution whenever one exists.
- **Optimal:** If it is guaranteed to find the best (lowest-cost) solution among all possible ones.
- **Uninformed (Blind) Search:** No additional information about states beyond the problem definition (start, goal, successors)
- **Informed (Heuristic) Search:** Uses the additional information about the estimated cost to goal



Uninformed Search Algorithms

- **Breadth-First Search (BFS):**

- Dequeues nodes in order of shallowest depth first (FIFO queue).

- **Depth-First Search (DFS):**

- Dequeues the most recently added node first (LIFO stack).

- **Iterative Deepening Search (IDS):**

- Repeatedly runs DFS, increasing the depth limit each time.

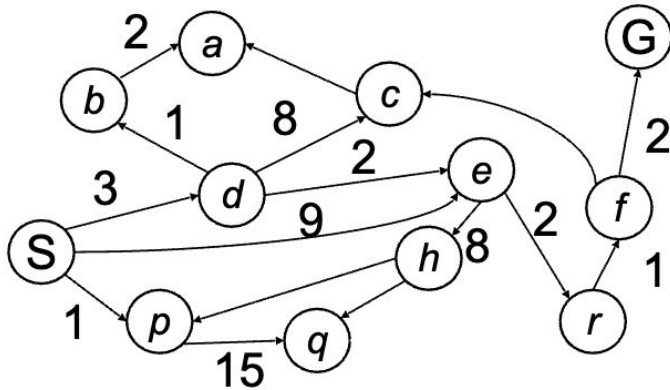
- **Uniform Cost Search (UCS):**

- Dequeues the node with the lowest cumulative path cost ($g(n)$) first.

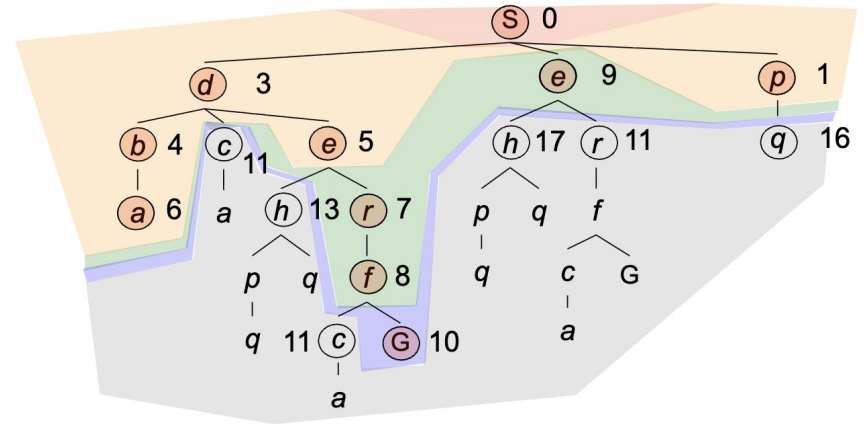


UCS Algorithm

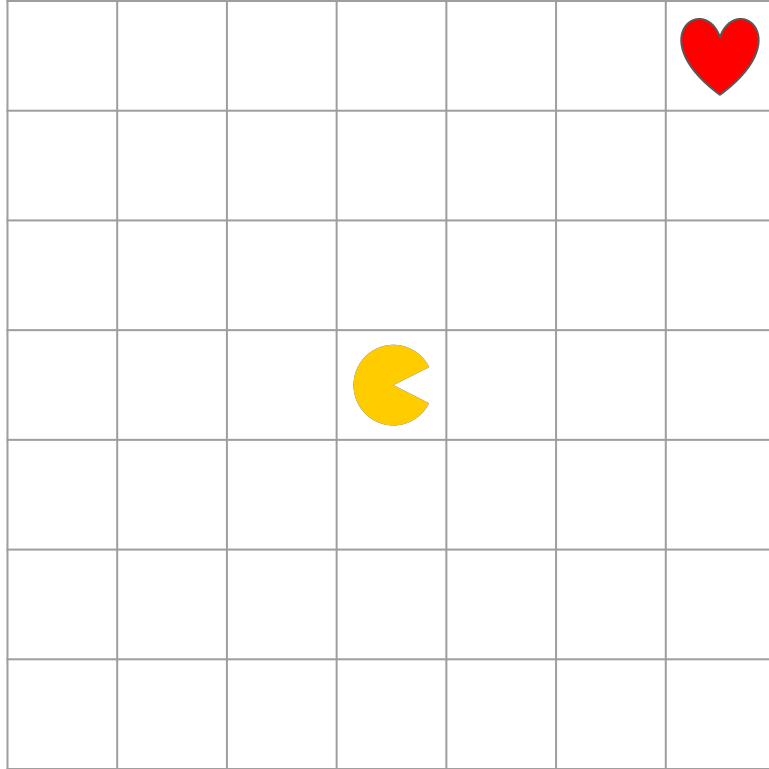
- $g(n)$ = cost from root to n
- Strategy: expand lowest $g(n)$
- Frontier is a priority queue sorted by $g(n)$



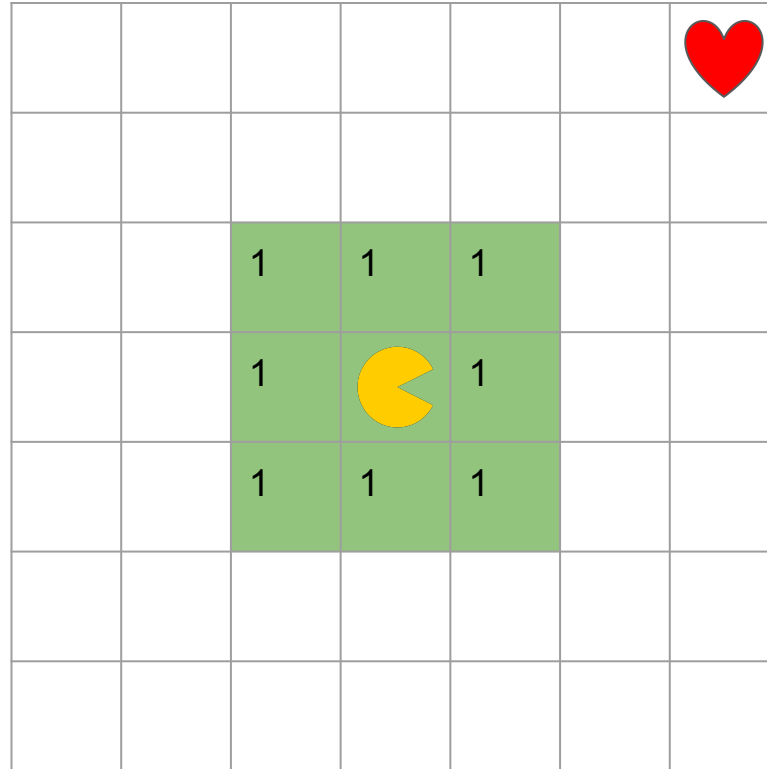
Cost
contours



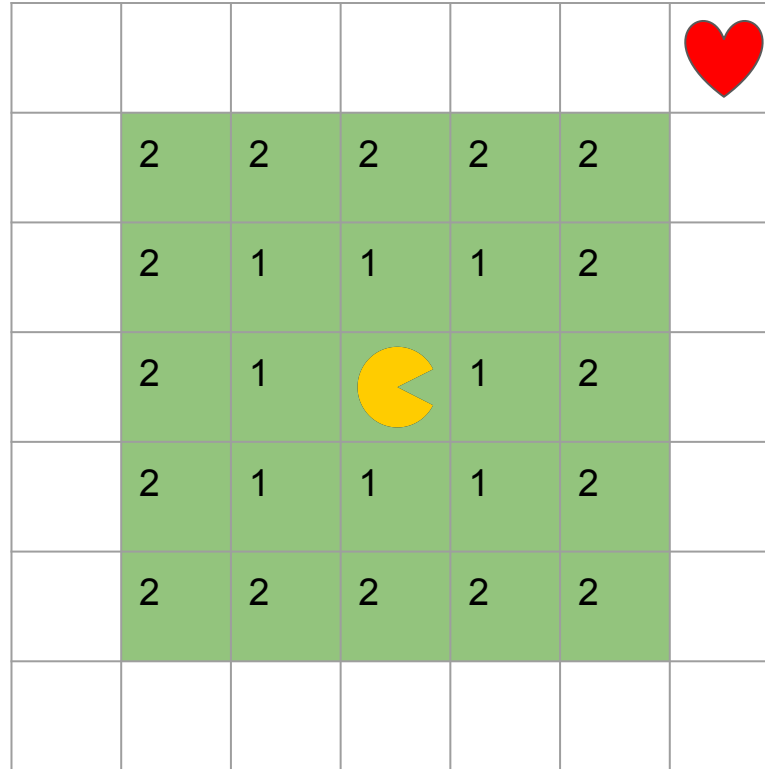
Why UCS is not Enough for us?



Why UCS is not Enough for us?



Why UCS is not Enough for us?



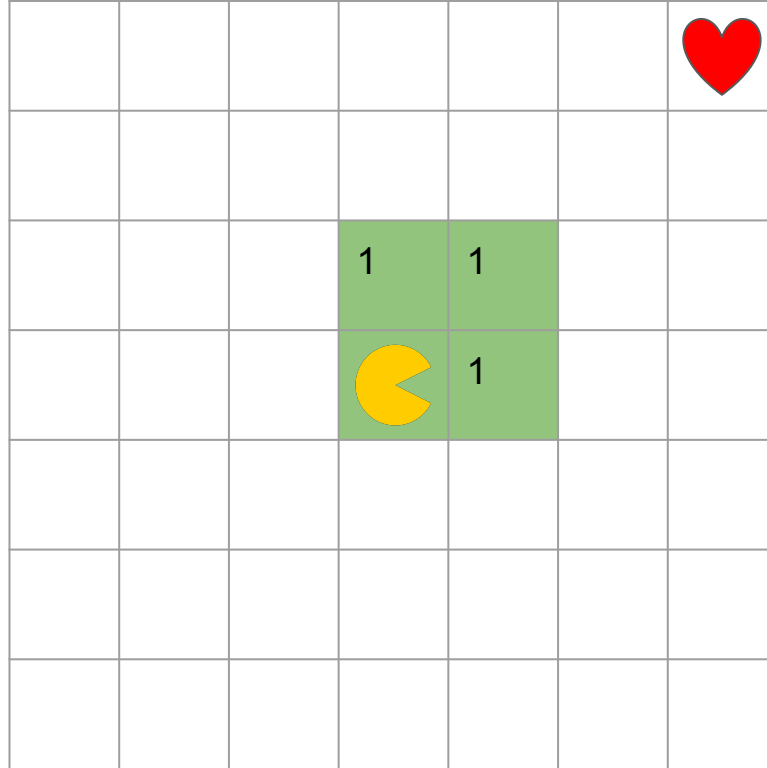
Why UCS is not Enough for us?

3	3	3	3	3	3	
3	2	2	2	2	2	3
3	2	1	1	1	2	3
3	2	1		1	2	3
3	2	1	1	1	2	3
3	2	2	2	2	2	3
3	3	3	3	3	3	3

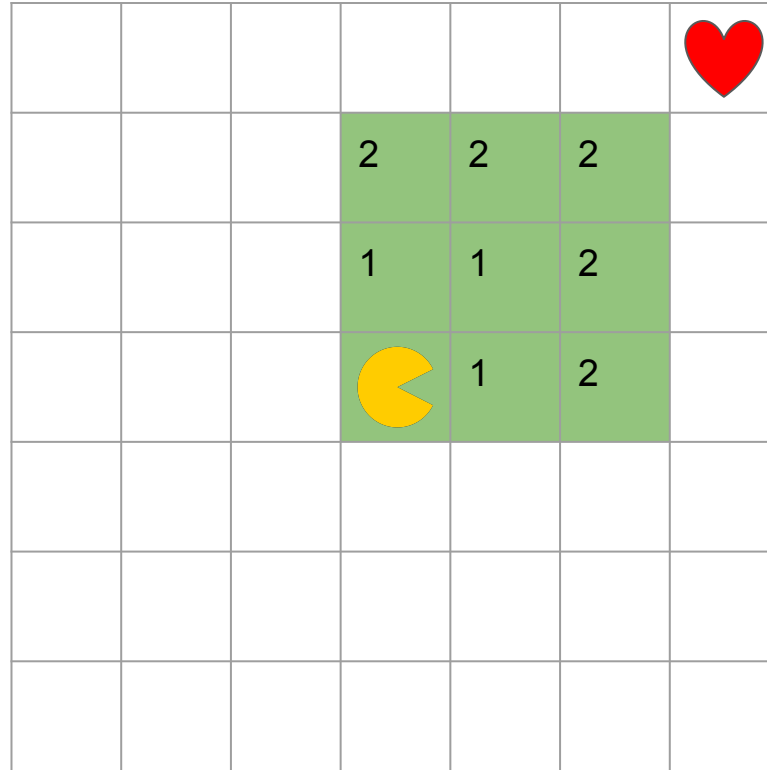
No information
about goal
location!



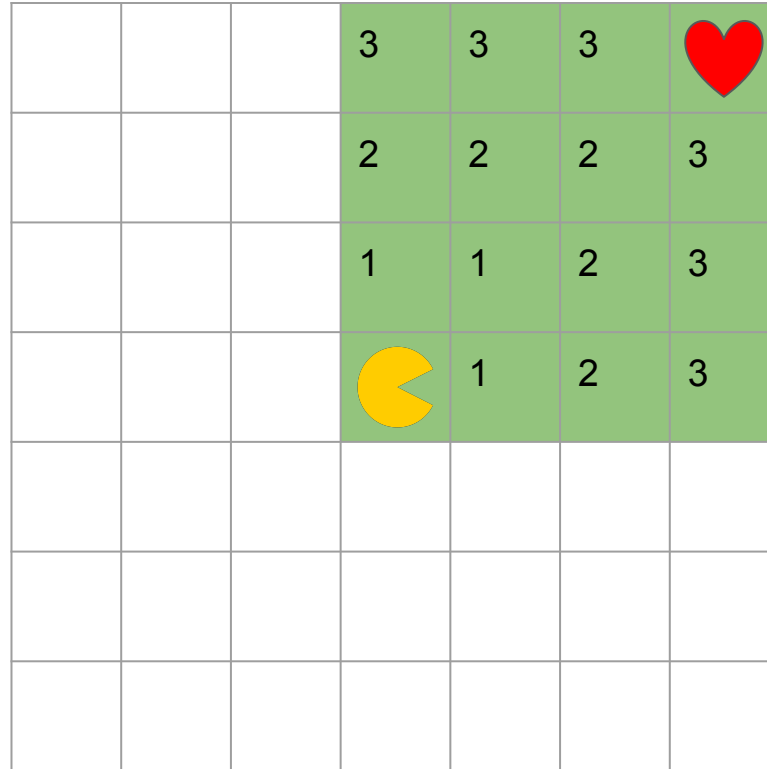
What does our intuition do?



What does our intuition do?



What does our intuition do?



UCS Explores Everywhere!

- UCS explores all plans with cost $< k$ before explore a plan of cost k



Uniform-Cost Search



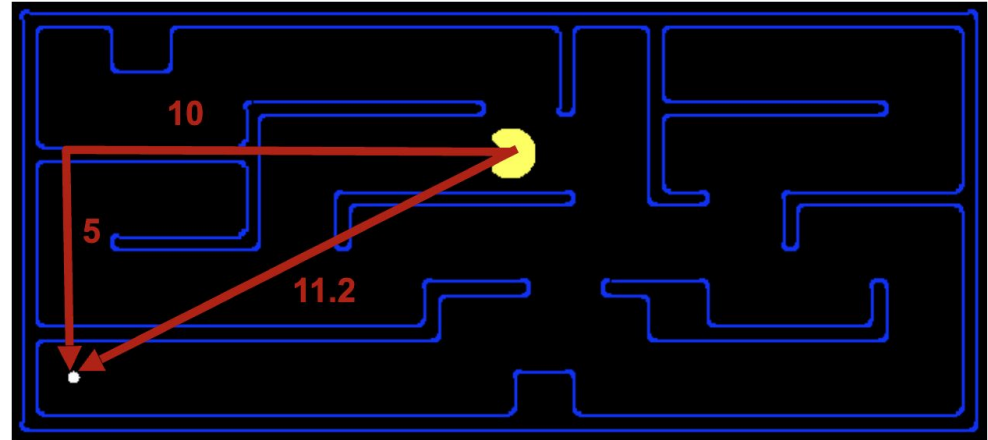
Informed Search

- Greedy
- A^*



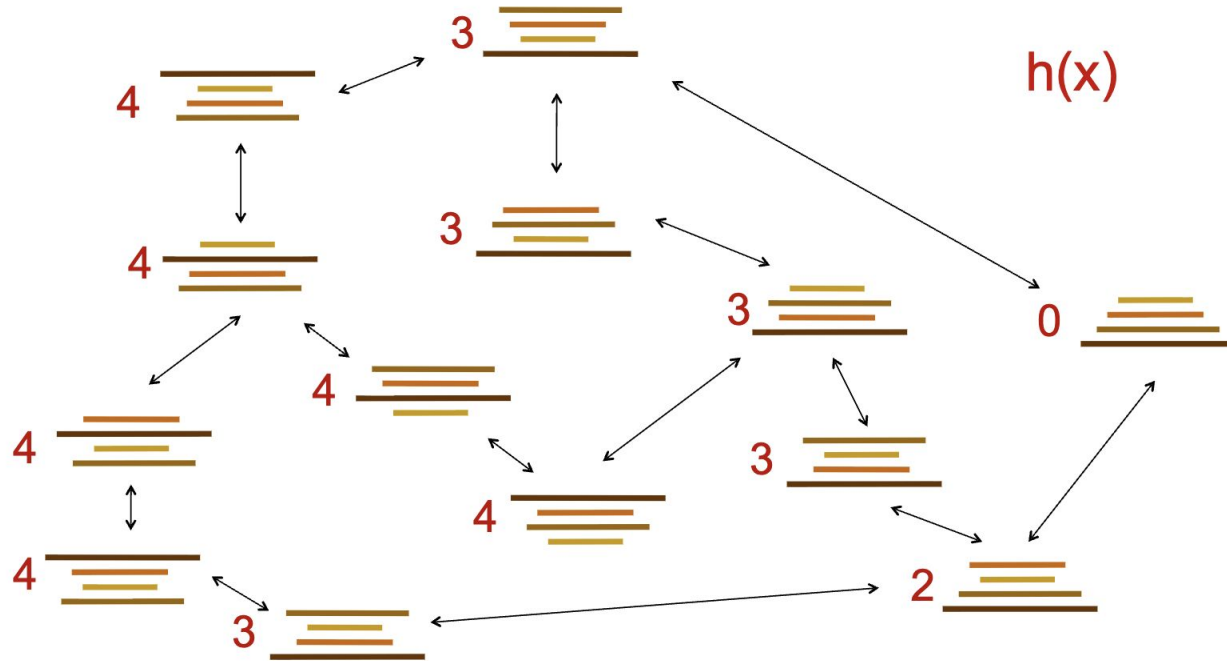
Search Heuristics

- A function $h(s)$: **estimates** how close a state is to a goal
- For each search problem we design/consider particular heuristics
- Example:
 - Manhattan distance
 - Euclidean distance



Search Heuristic - Pancake Example

- Heuristic: the number of the largest pancake that is still out of place



Search Heuristic

- Particular heuristics for different search problems
- What if the goal state changes?
- What if the start state changes?
- What if the cost between states changes?



Next Session: Informed Search

